

Governance Nodes Contract

Technical Specification

A single immutable smart contract for composable blockchain governance

Draft v1

~80 - 90% of a full, comprehensive spec

Buyer beware: might still contain a hallucination or two

Prepared for public implementation and open-source collaboration

by

Leo Glisic

May 4th, 2026

Overview

This document describes the Governance Nodes Contract, or GNC. The GNC is a single immutable smart contract that stores governance logic as state. It is meant to let people create governance systems by composing Nodes rather than by writing and deploying a custom governance contract for every new use case.

The document explains both what to build and why the design is shaped this way.

The style is intentionally practical. The goal is not to sound academic. The goal is for a builder to read this from front to back, understand the mental model, and then be able to implement a reference GNC in Vyper.

Terminology:

- GNC means the Governance Nodes Contract.
- Node means the one and only governance primitive stored in the GNC.
- NodeRef means a reference to a specific Node version, expressed as `NodeID.VersionPath`.

Table of Contents

- 1. The Core Concept
- 2. Design Philosophy
- 3. System Overview
- 4. Why One Contract
- 5. Why Vyper
- 6. The Public API
- 7. Node Identity and VersionPath
- 8. The Node Schema
- 9. Identity
- 10. Data
- 11. Clauses
- 12. Conditions and the Twelve Operators
- 13. Operands and References
- 14. Actions and the Four Action Types
- 15. Evaluation Semantics
- 16. Enactment Semantics
- 17. Caller and Payload
- 18. External Calls and External Contracts
- 19. Forking, Reuse, and Credit
- 20. The View Function
- 21. Events and Logs
- 22. Running Example
- 23. Formal Verification
- 24. Vyper Implementation Guidance
- 25. Edge Cases and Failure Modes
- 26. Testing Checklist
- 27. Reference Implementation Milestones
- 28. Glossary

1. The Core Concept

Most blockchain governance systems are built by writing custom smart contracts. A project wants voting, quorum, timelocks, delegation, vetoes, proposal schemas, execution delays, emergency powers, or upgrade rights, and the normal answer is to write or compose a new stack of contracts. That works, but it means the governance logic lives in bytecode. If the project wants to change the governance logic, the project often has to upgrade, redeploy, or wrap the contracts that define that governance system.

Governance Nodes takes a different approach. The GNC keeps the execution engine fixed, and moves governance logic into state. A governance rule becomes data. That data is stored inside Nodes. The GNC knows how to read those Nodes, evaluate their Clauses, and perform Actions when Conditions are met.

This means governance systems can be created, copied, forked, reused, and composed without writing a new governance contract every time. A quorum rule can be a Node. A timelock rule can be a Node. A veto rule can be a Node. A governor can be a Node that references those other Nodes. A project can reuse another project's rule, fork it, edit it, and credit the lineage on-chain through the version path.

The GNC is not trying to be a new general-purpose programming language. It is closer to a small governance instruction system. It has a fixed Node schema, a fixed set of twelve conditional operators, and a fixed set of four action types. Those constraints are not weaknesses. They are the reason the system can be understood, audited, composed, and eventually formally verified.

1.1 What problem the GNC solves

A smart contract can store state and execute code. In the classic upgrade pattern, a proxy contract stores state and points to an implementation contract. Governance decides when that pointer changes. The upgrade itself is simple: change a stored address. The hard part is deciding who is allowed to change it, under what conditions, after what delays, with what veto rights, and with what record of the decision.

Today those governance decisions are usually encoded in governance contracts. The GNC proposes a more general substrate: instead of writing a bespoke governance contract for every decision process, encode the process as Nodes inside a single contract that already knows how to evaluate governance logic.

The goal is not only to reduce deployment overhead. The deeper goal is reuse. Governance rules should be composable like software libraries. A well-designed veto process should be reusable. A well-designed vote threshold should be reusable. A well-designed emergency override rule should be reusable. If someone improves one, the improvement should be visible as a fork of the original, not as an unrelated contract with no lineage.

1.2 What the GNC does not try to solve

The GNC does not prevent people from creating bad governance. Ethereum lets people deploy bad contracts. The GNC likewise lets people create bad Nodes. A Node can be circular. A Node can be impossible to satisfy. A Node can call a dangerous external contract. A Node can permanently remove its own ability to change. That freedom is deliberate.

The GNC provides a clear execution model, not a nanny. It gives builders a minimal, expressive governance substrate. Safety profiles, conservative templates, formal verification tools, user interfaces, and best-practice libraries can grow around it. They should not be forced into the one immutable base contract.

2. Design Philosophy

The GNC is designed around five principles: simplicity at the base layer, maximum governance expressiveness, local sovereignty, composability through references, and a finite core that supports verification.

2.1 Simplicity at the base layer

The base contract should do very little, but do it exactly. It stores Nodes. It creates Nodes. It evaluates Nodes. It enacts Nodes. It exposes a view function so people can inspect Nodes. It does not include a separate admin policy contract, global banlist, global freeze switch, plugin registry, or upgrade path. If global safety machinery is desired, it should be built as Nodes or as tooling around Nodes, not hardwired into the GNC.

2.2 Maximum governance expressiveness

A governance system may need to call any contract, pass any calldata, read any value exposed by another contract, or use any social convention for how data is stored. The GNC should not decide that a project may only call approved targets or only use approved schemas. Those decisions belong to the Node author and the communities using those Nodes.

The GNC therefore treats Data as arbitrary structured information and treats Call Actions as governance-level calls with unrestricted calldata. If a project wants strict allowlists, caps, timelocks, or safer action wrappers, it can encode them in Nodes or use safer templates. The base protocol remains neutral.

2.3 Local sovereignty

Each Node governs itself. A Node can only edit itself, and it can only do so through its own Clauses. Node A cannot directly edit Node B. Node A may call Node B through the GNC, but Node B must decide for itself whether that call satisfies its Clauses. This is the core local sovereignty rule of Governance Nodes.

This rule keeps authority local. There is no hidden global admin. There is no universal judge deciding that one Node is safe and another is not. If a Node chooses to trust a caller, a contract, a payload format, or another Node, that trust is expressed in its own Clauses.

2.4 Composition through references

Nodes can reference other Nodes. A Governor Node can reference a Proposal Node. A Proposal Node can reference a Vote Count Node. A Vote Count Node can reference a Voter Eligibility Node. These references are explicit and versioned. Every Node reference identifies both the NodeID and the VersionPath.

This lets governance logic be reused without making special node types. There is no enforced Condition Node type or Action Node type. Those are human categories. A Node is just a Node. If its Data and Clauses are useful as a reusable condition, people may call it a Condition Node. If its Data and Clauses are useful as a reusable action, people may call it an Action Node. The GNC does not care.

2.5 A finite core for verification

The core has three Node fields, twelve conditional operators, and four action types. That finite vocabulary makes the GNC more tractable than an unrestricted on-chain scripting language. The GNC can be audited and reasoned about as the execution engine for a small governance calculus.

This does not mean every external call can be verified. If a Node calls an arbitrary external contract, the behavior of that external contract is outside the GNC's verification boundary. The GNC can verify that the decision process authorized the call. It cannot automatically verify the called contract unless that contract is separately verified.

3. System Overview

The GNC is a single smart contract. It stores every Node and contains the logic to create, evaluate, enact, and view those Nodes. The final architecture is one contract and one primitive: the Governance Nodes Contract and the Node.

Component	Meaning
Governance Nodes Contract (GNC)	The single immutable smart contract that stores Nodes and executes the Governance Nodes logic.
Node	The primitive object stored in the GNC. Every Node has Identity, Data, and Clauses.
Identity	The identifying and lineage metadata of the Node, including NodeID and VersionPath.
Data	The Node's arbitrary values and state. Data may be structured however the Node author chooses.
Clauses	Ordered IF/THEN rules. Each Clause has a Condition and one or more Actions.
Condition	A formula-like expression built from the twelve canonical operators and operands.
Action	One of four operation types: Return, Call, Edit, or Emit.

3.1 The canonical lifecycle

1. A user creates a Node by calling `createNode`. The Node receives an Identity, Data, and Clauses.
2. A user or another Node can inspect a Node by calling `viewNode`.
3. A user or another Node can evaluate a Node by calling `evaluateNode`. This reads the Node and returns what its Clauses currently say, without changing state.
4. A user or another Node can enact a Node by calling `enactNode`. This evaluates the Clauses and executes Actions for every Clause whose Condition is true.
5. If an Action edits the Node, the edit can only apply to that same Node. A Node cannot directly edit another Node.
6. If an Action calls another Node, that other Node decides for itself whether to act based on its own Clauses.

3.2 The minimum public surface

The GNC has three core governance functions and one generalized view function:

Function	Purpose	State-changing?
<code>createNode</code>	Creates a new Node or forks an existing Node into a new VersionPath.	Yes
<code>evaluateNode</code>	Evaluates a Node's Clauses and returns a read-only result.	No
<code>enactNode</code>	Evaluates a Node's Clauses and executes Actions for true Clauses.	Yes
<code>viewNode</code>	Reads any requested part of a Node.	No

The first three are the conceptual core. The view function exists because real UIs, indexers, wallets, and external contracts need a clean way to inspect Node contents. It is not a governance function; it is an inspection function.

4. Why One Contract

A multi-contract architecture can be attractive at first. One contract might store Nodes, another might interpret Conditions, another might execute Actions. That architecture gives modularity, but it also introduces extra trust boundaries, version mismatch problems, and more places for governance logic to hide.

The GNC deliberately keeps the system inside one immutable contract. The contract code is the execution engine. The Nodes are the source of governance logic. If a community wants different behavior, it creates different Nodes. If someone wants different base semantics, they can deploy a different GNC. The canonical GNC remains small, fixed, and knowable.

This matters for public-good governance infrastructure. If many projects are going to rely on one base contract, the contract should be as boring and permanent as possible. The complexity should live in state where it can be inspected, copied, and forked. The execution semantics should not be a moving target.

4.1 Immutability is a feature, not a bug

The GNC should not be upgradeable. Upgradeability at the base layer would reintroduce the governance problem the GNC is meant to simplify. If the execution engine can change underneath everyone, then the meaning of every Node can change. That destroys the main benefit of storing governance logic as data inside a known immutable contract.

Future versions of the GNC can be deployed. They can import or recreate Nodes if communities want to migrate. But the original contract should remain a stable public object. A Node that points to GNC address X should always mean the same GNC semantics at address X.

4.2 No hidden administrator

The GNC must not have an owner with power to freeze Nodes, ban callers, upgrade logic, or change operator semantics. Any such power should be expressed as a Node and voluntarily adopted by other Nodes through Clauses. This keeps the base layer neutral.

5. Why Vyper

The reference implementation is expected to use Vyper. The main reason is not branding, taste, or because Solidity cannot do the job. Solidity can implement almost anything on the EVM. The reason is that the GNC stores mutable, forkable, self-editing Node records, and the implementation must be especially careful about complex data structures, copying, editing, and deletion-like behavior.

The GNC will copy Nodes when they are forked. It will edit fields inside Nodes. It may clear or replace Data fields or Clauses. It must never accidentally make two Node versions share a mutable internal reference. It must never treat array position as identity. It must never let deleting or shortening one data structure disturb references held by other Nodes.

At the EVM level, storage is key-value slot based. The EVM does not move storage around the way a normal runtime might move objects in memory. The risk is not that the EVM will literally relocate a Node. The risk is at the language and implementation layer: accidental aliasing, shallow-copy mistakes, storage pointer confusion, dynamic array

position being confused with canonical identity, or a data structure being edited while another part of the code assumes it is stable.

Vyper's stricter approach to complex data structures, boundedness, reference handling, and auditability makes it a better fit for a contract whose entire job is to manage structured governance state safely. The design must still be careful. Vyper is not magic. But Vyper pushes the implementation toward explicit storage layouts and away from clever pointer-like behavior.

Implementation principle: Node references MUST be canonical identifiers, not memory positions or array indexes. A NodeRef is NodeID.VersionPath. Internal storage can use mappings, counters, and indexes, but those are implementation details.

5.1 Avoiding shallow-copy forks

Forking is central to the GNC. When a Node is forked, the new version must be an independent copy. If the parent later edits its Clauses, the child must not silently change. If the child edits its Data, the parent must not silently change. A fork is a deep copy of the Node's Identity-derived lineage, Data, and Clauses, with a new VersionPath assigned by the GNC.

A reference implementation should therefore avoid storing nested dynamic structures in a way that makes deep copying ambiguous. A robust pattern is to store Node fields in mappings keyed by NodeRef-derived keys, and to copy field records explicitly during a fork. Another robust pattern is to store content-addressed chunks and treat shared chunks as immutable. What the implementation must not do is create a child version that points to mutable parent-owned records unless that sharing is immutable by construction.

5.2 Deletion and deprecation

The core GNC does not need a public deleteNode function. Deleting a canonical NodeID.VersionPath would break references and weaken lineage. If a Node should no longer be used, it can mark itself as deprecated in Data, clear or replace its Clauses, or emit an event. Optional fields can be cleared if the Node's own Clauses allow it. But the canonical Identity record should remain inspectable.

In other words, Nodes can evolve and can even make themselves unusable, but history should remain visible. A public governance substrate should prefer tombstones and lineage over disappearing records.

6. The Public API

This section defines the conceptual API. Exact Vyper signatures may vary, especially around how arbitrary structured Data, formulas, and actions are encoded. The behavior is normative even when the precise ABI representation is left to the implementation.

6.1 createNode

createNode creates either a new Node lineage or a fork of an existing Node version.

```
createNode(createMode, sourceNodeRef?, identityInput, dataInput, clausesInput) ->
NodeRef
```

There are two creation modes:

Mode	Meaning
------	---------

NEW	Create a new NodeID with VersionPath 1. The caller supplies initial Identity metadata, Data, and Clauses.
FORK	Copy an existing NodeID.VersionPath. The GNC preserves the same NodeID and assigns the next available child VersionPath under the source version.

In NEW mode, the GNC assigns a new NodeID and sets the VersionPath to 1. In FORK mode, the GNC copies the source Node exactly, except for GNC-maintained Identity fields such as VersionPath, creator, creation metadata, and parent lineage. The fork preserves the same NodeID because this is a lineage relationship. A fork is a citation. Creating a visually identical Node under a fresh NodeID is technically possible, but the protocol should make proper lineage the easy and honorable path.

6.2 evaluateNode

evaluateNode reads a Node, evaluates its Clauses in order, and returns a structured read-only result. It must not edit storage. It must not execute state-changing external calls. It may perform read-only external calls needed by Conditions.

```
evaluateNode(nodeRef, payload) -> EvaluationResult
```

The evaluation result SHOULD include, at minimum, the truth value of each evaluated Clause and any Return Action outputs for true Clauses. It MAY include additional diagnostic information if the implementation supports it.

6.3 enactNode

enactNode evaluates a Node's Clauses in order and executes Actions for every Clause whose Condition evaluates to true.

```
enactNode(nodeRef, payload) -> EnactmentResult
```

The GNC MUST process Clauses deterministically in their declared order. Within each true Clause, the GNC MUST process Actions in their declared order. If an Action fails, the GNC handles the failure according to that Action's onFailure flag.

enactNode returns a structured result containing Return Action outputs and execution status information. It also emits events for observability.

6.4 viewNode

viewNode is a generalized read function. It exists because a single canonical inspection function is cleaner than a long list of specialized getters.

```
viewNode(nodeRef, query) -> bytes
```

The query parameter specifies what to read. For example, a caller may request the full Node, only Identity, only Data, a Data path, all Clauses, one Clause, one Action, or lineage metadata. The returned bytes should be encoded in a documented format.

This is not a governance action. It is an inspection helper for users, indexers, wallets, explorers, and other contracts.

7. Node Identity and VersionPath

Every Node is identified by a pair: NodeID and VersionPath. A Node reference is not just a NodeID. It is always NodeID.VersionPath. This is essential because Nodes can fork. Two versions of the same NodeID may have different Data and Clauses.

```
NodeRef = NodeID.VersionPath
Example: 42.1
Example: 42.2
Example: 42.2.1
```

7.1 NodeID

NodeID is numerical and canonical. It should not be a human-readable string. A Node may include a human-readable name or label in Identity, but that label is not unique and is not canonical. Names are for people. NodeID is for the contract.

7.2 VersionPath

VersionPath is an ordered list of integers. It is displayed with dots, but it is not a decimal number. VersionPath 2.10 is the path [2, 10], not the decimal value 2.1. This matters because the dots show lineage.

VersionPath	Meaning
1	The first version of a newly created NodeID.
2	A later direct version under the same NodeID, if created by the GNC as such.
2.1	The first fork of version 2.
2.2	The second fork of version 2.
2.1.1	The first fork of version 2.1.

When a Node is forked, the GNC assigns the next available child version under the source VersionPath. If 42.2 already has children 42.2.1 and 42.2.2, the next fork of 42.2 becomes 42.2.3.

7.3 Why visible lineage matters

VersionPath is partly technical and partly cultural. Governance Nodes are meant to be copied, reused, and improved. A good Node may become a public-good building block. When someone forks it, the lineage should be visible. This is similar to citations in academia or forks in open-source software. The protocol cannot force good manners, but it can make good manners easy.

8. The Node Schema

The Node schema is fixed. Every Node has exactly three top-level fields:

7. Identity
8. Data
9. Clauses

This simplicity is the heart of the design. Authority is expressed in Clauses. Conditions live inside Clauses. Actions live inside Clauses. Versioning and lineage live in Identity. Arbitrary state and contextual information live in Data. The fixed three-field schema is what makes the system easy to explain while still leaving almost unlimited room for composition.

Field	Purpose	Editable by Node?
Identity	Canonical identity, lineage, creator metadata, and GNC-maintained metadata.	Partly. NodeID and VersionPath are GNC-only. Other Identity metadata may be editable if allowed by Clauses.
Data	Arbitrary values and state used by the Node.	Yes, through the Node's own Clauses and Actions.
Clauses	Ordered IF/THEN rules that define when and how the Node acts.	Yes, through the Node's own Clauses and Actions.

8.1 No enforced Node types

There are no special Node classes in the GNC. A Condition Node is just a Node whose Data and Clauses are useful as a condition. An Action Node is just a Node whose Data and Clauses are useful as an action. A Governor Node is just a Node that governs something. These labels are helpful for humans, but the GNC does not enforce them.

This is important because future users will invent Node categories nobody can predict today. The base contract should not limit the imagination of future governance designers.

9. Identity

Identity is the part of a Node that tells the world what the Node is, where it came from, and how to refer to it. Some Identity fields are controlled only by the GNC. Others may be user-supplied or editable through the Node's own Clauses.

Identity field	Description	Who controls it?
nodeId	Numerical canonical ID for the Node lineage.	GNC only
versionPath	Ordered integer path identifying this version/fork.	GNC only
parentNodeId	NodeID of the parent version if this Node was forked.	GNC only
parentVersionPath	VersionPath of the parent if this Node was forked.	GNC only
creator	Address that created this version.	GNC sets at creation
createdBlock / createdTime	Creation metadata.	GNC only
lastEditedBlock / lastEditedTime	Last time this Node was changed.	GNC only
name	Optional human-readable name.	May be editable by the Node if its Clauses allow it

description / metadataURI	Optional human-readable description or metadata pointer.	May be editable by the Node if its Clauses allow it
---------------------------	--	---

The GNC MUST prevent a Node from editing its own `nodeId` and `versionPath`. Those fields define the canonical identity of the record. If they were editable, references would become unreliable. A Node may edit descriptive Identity fields only if its own Clauses allow such edits.

10. Data

Data is the Node's arbitrary state. It can contain booleans, integers, addresses, strings, bytes, lists, structs, or any other values an implementation supports. The GNC should treat Data as structured enough for Conditions and Actions to reference fields, but flexible enough that Node authors can create their own schemas.

A useful mental model is that Data is a struct-like object whose schema is chosen by the Node author. One Node may store a proposal status, vote counts, and a target contract. Another may store a reusable threshold formula. Another may store only a human-readable note. The GNC does not impose one universal Data schema.

10.1 Why not one Solidity-style struct for Data?

A fixed Solidity or Vyper struct is excellent when the contract knows all fields at compile time. The GNC does not know all future governance designs at compile time. Therefore Data cannot be one fixed struct shared by every Node. Instead, the implementation should support arbitrary structured data in a way that Conditions can reference.

A reference implementation may represent Data as typed fields keyed by field identifiers or paths. It may also support ABI-encoded structs as values. The conceptual requirement is simple: Data must be able to represent arbitrary governance-relevant values, and Clauses must be able to reference those values.

10.2 Suggested Data value model

The conceptual spec does not force a single storage encoding, but a practical reference implementation should define a tagged value model. For example, each Data value may have a type tag and encoded bytes.

Type	Example use
BOOL	<code>passed = TRUE, vetoed = FALSE</code>
UINT	<code>yesVotes = 73, quorumRequired = 1000</code>
INT	signed deltas or negative values, if supported
ADDRESS	token contract, treasury contract, proxy contract
BYTES32	hashes, identifiers, merkle roots
BYTES	arbitrary ABI-encoded payloads
STRING	human-readable labels or explanations
NODE_REF	reference to another <code>NodeID.VersionPath</code>
LIST / STRUCT	implementation-defined compound data

10.3 Undefined data

Conditions need a clean way to reason about missing fields. The `IS_DEFINED` operator exists for this reason. If a Condition tries to compare an undefined value without first checking it, the implementation **SHOULD** treat that comparison as false or revert according to a documented rule. The recommended rule is: `IS_DEFINED` returns false for missing values; other operators on missing values fail the containing Condition unless wrapped in logic that handles absence.

11. Clauses

Clauses are the executable governance rules of a Node. A Clause has the shape of an IF/THEN statement. If its Condition is true, its Actions run. If its Condition is false, the Clause is skipped.

```
IF(condition, THEN(action_1, action_2, ...))
```

This format is meant to feel closer to Excel than to a programming language compiler. People understand formulas. Governance Clauses should be readable as formulas: IF these things are true, THEN do these things.

11.1 Clause fields

Clause field	Meaning
clauseId	Stable identifier for the Clause within the Node. It should not change merely because the display order changes.
order	The Clause execution order. Clauses are evaluated from lowest order to highest order.
condition	Formula-like expression that evaluates to TRUE or FALSE.
actions	Ordered list of Actions to execute if condition is TRUE.
metadata	Optional descriptive metadata, comments, labels, or UI hints.

clauseId should be stable. If a Node edits its Clauses, a returned value should be able to say Clause 3 returned 56, and that should refer to a stable Clause identity, not whatever happens to be at array index 3 after an edit.

11.2 Clause execution order

When `enactNode` runs, the GNC evaluates all Clauses in order. It does not stop after the first true Clause. Every Clause whose Condition is true has its Actions executed. This lets a Node update multiple fields, emit multiple logs, call multiple external contracts, and return multiple values in one enactment.

If a Clause edits a later Clause, the implementation must define whether that edit affects the current enactment or only future enactments. The recommended rule for a simple reference implementation is live sequential semantics: the GNC reads each Clause when it reaches that Clause. If an earlier Action edits a later Clause before the later Clause is reached, the later Clause is evaluated as edited. Already evaluated Clauses are not re-run. Authors who want stable behavior should put Clause-editing Actions after Actions that depend on the old Clause set.

This rule is expressive and close to normal EVM behavior. A more conservative implementation could snapshot Clauses at the start of enactment, but that requires more copying and may be harder to implement in Vyper. The canonical v1 semantics are live sequential semantics unless the reference implementation explicitly documents snapshot semantics.

11.3 IF/THEN formula examples

```
IF(
  GT(Self.Data.yesVotes, 2),
  THEN(
    EDIT(Self.Data.vetoStartTime, BLOCK_TIMESTAMP),
    EMIT("Veto period started")
  )
)

IF(
  AND(
    GT(Self.Data.yesVotes, 2),
    EQ(Node[77.1].Data.vetoes, 0),
    GT(BLOCK_TIMESTAMP, Self.Data.vetoStartTime + 30 days)
  ),
  THEN(
    EDIT(Self.Data.implementationStartTime, BLOCK_TIMESTAMP)
  )
)
```

The plus sign and days notation in the example are author-facing sugar. The canonical operator set is still the twelve operators listed later. A reference implementation may include helper encoding for time arithmetic, or it may require authors to precompute timestamps and compare integers directly.

12. Conditions and the Twelve Operators

A Condition is a formula that evaluates to TRUE or FALSE. The v1 GNC has exactly twelve conditional operators. This operator set is intentionally small. It covers equality, ordering, membership, existence, and boolean composition.

Operator	Meaning	Operand types
$X = Y$	True if X equals Y.	Any comparable type
$X \neq Y$	True if X does not equal Y.	Any comparable type
$X < Y$	True if X is less than Y.	Ordered numeric or comparable values
$X > Y$	True if X is greater than Y.	Ordered numeric or comparable values
$X \leq Y$	True if X is less than or equal to Y.	Ordered numeric or comparable values
$X \geq Y$	True if X is greater than or equal to Y.	Ordered numeric or comparable values
$X \text{ IN } Y$	True if X is included in list/set Y.	Element and collection
$Y \text{ CONTAINS } X$	True if collection Y contains X.	Collection and element
$X \text{ IS_DEFINED}$	True if X exists and resolves to a value.	Any reference
$X \text{ AND } Y$	True if both X and Y are true.	Boolean
$X \text{ OR } Y$	True if either X or Y is true.	Boolean

NOT X	True if X is false; false if X is true.	Boolean
-------	---	---------

12.1 Formula syntax versus storage encoding

The human-readable syntax can look like Excel functions or spreadsheet formulas. The on-chain encoding should be deterministic and compact. These are two layers of the same idea.

Human-readable	Canonical idea
AND(A, B)	Boolean operator with two child expressions.
OR(AND(A, B), C)	Nested expression tree.
GT(Node[12.1].Data.yesVotes, 2)	Comparison between a Node Data reference and a literal.
IS_DEFINED(Self.Data.vetoStartTime)	Existence check for a Data path.

The GNC should not depend on parsing human text directly on-chain. Human-friendly tooling can compile formula-like authoring syntax into the canonical on-chain Condition encoding.

12.2 Expression trees

Conditions should be represented as expression trees, not ambiguous flat strings. The expression tree for OR(AND(A, B), C) has OR at the root, AND as the left child, and C as the right child. This avoids precedence confusion. Nobody has to wonder whether A AND B OR C means (A AND B) OR C or A AND (B OR C). The tree says exactly what is meant.

The author-facing formula can still be easy to read. It just compiles into a tree. This is how many spreadsheet formulas and programming language expressions already work.

12.3 Type errors

The GNC must define what happens when an operator receives values of incompatible types. The recommended rule is strict: incompatible operands cause the Condition evaluation to fail. In `evaluateNode`, the result should indicate the failure. In `enactNode`, the failed Condition should evaluate to false unless the implementation chooses to revert on malformed Conditions. For public-good safety, malformed Clauses should be visible and diagnosable rather than silently behaving in surprising ways.

13. Operands and References

An operator needs operands. An operand is a value source. It can be a literal value, a field inside the current Node, a field inside another Node, a value from the payload, the immediate caller, block context, or a value read from an external smart contract.

Operand source	Example	Notes
Literal	TRUE, 7, 0xabc..., bytes32 hash	Direct value encoded in the Condition.
Self Identity	Self.Identity.versionPath	Reads Identity of current Node.

Self Data	Self.Data.yesVotes	Reads Data from current Node.
Self Clause	Self.Clauses[clauseId]	Can inspect Clause data if implementation exposes it.
Other Node	Node[42.2.1].Data.passed	Every Node reference includes NodeID and VersionPath.
Caller	CALLER	Immediate caller to this Node: wallet, contract, or NodeRef.
Payload	PAYLOAD.path	Data supplied with evaluateNode or enactNode.
Block context	BLOCK_NUMBER, BLOCK_TIMESTAMP, CHAIN_ID	Standard chain context.
External contract read	READ(contract, calldata, returnType)	Read-only call to another contract.

13.1 NodeRef operands

A NodeRef is always NodeID.VersionPath. A Condition should not reference only a NodeID because different versions may have different meanings. External contracts should also pin authority to a full NodeRef, not just a NodeID.

13.2 Caller operands

A Node receives a caller. The caller may be a wallet address, a smart contract address, or another Node that called it through the GNC. The GNC should expose this caller value to Conditions. If a Node wants to preserve a longer call chain, it can pass that history forward in the payload. The base GNC does not need to maintain a complex execution context.

This keeps the protocol simple. A Node that cares only about its direct caller can check CALLER. A Node that cares about deeper provenance can require the caller to pass that information in payload and verify it through Clauses.

13.3 External contract reads

Conditions must be able to evaluate smart contract state outside the GNC. The EVM does not let one contract directly inspect arbitrary storage slots of another contract. But it does let a contract call another contract. Therefore external reads should be represented as read-only calls to target contracts that expose view functions.

If a target contract does not expose the needed value, a separate adapter contract can expose it. The GNC does not need to know whether the target is a token, oracle, voting contract, delegation registry, or adapter. It only needs a deterministic call, calldata, and expected return encoding.

14. Actions and the Four Action Types

If a Clause's Condition is true, the GNC executes that Clause's Actions in order. v1 has exactly four action types:

10. Return value
11. Call function
12. Edit field / data
13. Emit event / log

Action type	Purpose	Can change state?
RETURN	Add a value to the Node's returned result.	No
CALL	Call a GNC function or an external smart contract function with arbitrary calldata.	Yes, if target call changes state
EDIT	Edit a field of the current Node.	Yes
EMIT	Emit an event/log from the GNC.	No storage change, but emits log

14.1 Return value

A Return Action appends a structured return record to the result of `evaluateNode` or `enactNode`. It does not halt execution. This is important because the GNC evaluates all Clauses in order, and multiple Clauses may return useful values.

Every return record MUST include the `clauseId` that produced it. A UI may display returns as a semicolon-separated string, but the contract-level return should be structured.

```
Return records:
[
  { clauseId: 3, value: 56 },
  { clauseId: 7, value: TRUE },
  { clauseId: 9, value: "approved" }
]
```

14.2 Call function

A Call Action calls a function. The target may be the GNC itself or any external smart contract. The calldata is unrestricted. This is required for full governance-level expressiveness. A governance decision might need to call a proxy upgrade function, transfer funds from a treasury, call another Node through `enactNode`, call a voting contract, or trigger a cross-chain message bridge.

The GNC should not impose a global allowlist of call targets. If a Node wants an allowlist, it should encode that allowlist in its own Data and Clauses. The base contract should not decide what governance is allowed to call.

A Call Action that calls another Node does not directly edit that other Node. It calls the GNC, and the target Node decides whether to edit itself through its own Clauses. This preserves local sovereignty.

14.3 Edit field / data

An Edit Action edits a field of the current Node. This may include Data, Clauses, and editable Identity metadata. It may not edit the `NodeID` or `VersionPath`, because those are GNC-controlled identity fields.

This is how governance over governance works. A Node can contain a Clause saying that if a certain proposal passes, then the Node edits one of its own Clauses. That is not a special upgrade mechanism. It is simply an Edit Action authorized by the Node's own IF/THEN logic.

14.4 Emit event / log

An Emit Action emits a log from the GNC. Events are important because off-chain systems need to understand what happened. A Node can emit simple events like ProposalPassed, VetoStarted, or UpgradeExecuted, or it can emit arbitrary bytes defined by the Node author.

Events do not persist if the transaction reverts. This is normal EVM behavior. If a user wants a diagnostic log of a failed enactment, the user should first call evaluateNode or use tooling that simulates enactNode off-chain.

14.5 Action failure behavior

Each Action has an onFailure flag. v1 has exactly two failure modes:

Value	Name	Behavior
0	SKIP	If this Action fails, skip it and continue to the next Action. This is the default.
1	REVERT	If this Action fails, revert the entire enactNode transaction.

There is intentionally no 'revert only this Clause' mode in v1. The EVM naturally supports reverting the whole transaction, not selectively rolling back arbitrary external calls from one Clause while keeping later Clauses alive. A partial rollback mode would create false expectations, especially when Actions can call arbitrary external contracts.

15. Evaluation Semantics

evaluateNode is the read-only way to ask, 'What does this Node say right now?' It evaluates Conditions and produces return information, but it does not perform state-changing Actions.

15.1 Evaluation order

14. Load the target Node by NodeID.VersionPath.
15. Read the caller and payload.
16. Evaluate Clauses in order.
17. For each Clause, evaluate its Condition.
18. If the Condition is true, process read-only consequences such as Return Action outputs. Do not perform CALL, EDIT, or EMIT as state-changing effects.
19. Return a structured EvaluationResult.

evaluateNode should be safe to call from wallets, UIs, indexers, and external contracts. It may perform read-only external calls needed by Conditions. It must not write to the GNC. It must not emit events, because a true EVM view/static context cannot emit logs.

15.2 What evaluateNode returns

A good EvaluationResult should be useful to humans and machines. It SHOULD include:

- the NodeRef evaluated;
- the payload hash or payload echo if appropriate;
- each Clause evaluated and whether its Condition was true;
- Return Action outputs for true Clauses;
- diagnostic errors for malformed Conditions, undefined references, or type mismatches if the implementation can provide them.

15.3 Evaluation is not a promise of enactment

A true evaluation does not guarantee a later enactment will succeed. State may change between the evaluation transaction and the enactment transaction. External contract reads may change. A Call Action may fail. Gas may run out. The evaluation is a snapshot of what the Node says when evaluated, not a reservation of future state.

16. Enactment Semantics

enactNode is the state-changing way to ask a Node to act. It evaluates Clauses in order and executes Actions for every true Clause.

16.1 Enactment algorithm

20. Load the target Node by NodeID.VersionPath.
21. Determine the caller: wallet, smart contract, or NodeRef if called from another Node through the GNC.
22. Attach the payload supplied by the caller.
23. Evaluate Clause 1. If false, skip it. If true, execute its Actions in order.
24. Continue with each later Clause in order.
25. If an Action fails with onFailure = SKIP, continue. If it fails with onFailure = REVERT, revert the entire enactNode transaction.
26. Append Return Action outputs to the result as they occur.
27. Emit execution events as appropriate.
28. Return the EnactmentResult.

16.2 Self-editing rule

The most important enactment rule is this:

Self-editing rule: A Node may edit any editable field of itself through its own Clauses and Actions. A Node may not directly edit another Node.

This rule is simple but powerful. It means Node B's security is always defined by Node B. Node A can call Node B, but B decides whether that call matters. A wallet can call Node B, but B decides whether that wallet can cause an edit. A smart contract can call Node B, but B decides whether that contract and payload satisfy its Clauses.

16.3 Calling another Node

To make one Node interact with another, the first Node uses a Call Action targeting the GNC's enactNode or evaluateNode function for the second Node. The GNC then invokes the second Node with caller set to the calling Node. The second Node may check CALLER in its Conditions.

```
Node A action:
CALL GNC.enactNode(NodeB.2.1, payload)

Node B condition:
IF(EQ(CALLER, NodeA.3), THEN(EDIT(Self.Data.received, TRUE)))
```

This is how Nodes compose without violating self-sovereignty.

16.4 Reentrancy and nested calls

Because Call Actions can call arbitrary contracts, and because those contracts may call back into the GNC, implementations must handle reentrancy deliberately. The spec does not forbid reentrancy because recursive governance composition may be useful. Instead, the implementation must make reentrant behavior explicit and safe.

At minimum, the GNC should ensure that storage updates are internally consistent before making external calls, or it should clearly document the ordering if external calls are made before edits. The simplest authoring guideline is: edit internal Node state before making external calls if the external call depends on the edited state; use `onFailure = REVERT` if partial execution would be unsafe.

A reference implementation may include a call-depth counter to avoid accidental infinite recursion, but the conceptual protocol treats gas as the practical limit. If an author creates a recursive loop that runs out of gas, that is a bad Node, not a failure of the GNC.

17. Caller and Payload

Every evaluation or enactment has a caller and a payload. The caller is the immediate entity asking this Node to do something. It may be a wallet address, a smart contract address, or another NodeRef. The payload is arbitrary bytes supplied by the caller.

17.1 Caller

The GNC should expose CALLER as an operand that Conditions can inspect. If a wallet calls the Node directly, CALLER is the wallet. If a smart contract calls the Node directly, CALLER is that contract. If another Node calls this Node through the GNC, CALLER is the calling NodeRef.

This avoids a large execution context object. If deeper call history matters, the caller can include it in payload. A Node can then require that payload to contain whatever history it wants.

17.2 Payload

Payload is freeform. It can contain a vote, a signature, an external calldata message, a proposed Data edit, a serialized call chain, a human-readable note, or any encoding a Node author chooses. The GNC should not impose a universal payload schema.

If a Condition or Action expects a payload field, that expectation should be encoded in the Node's Clauses and Data conventions. Tooling can help authors compile human-readable payload schemas into canonical encodings, but the GNC should remain neutral.

17.3 Passing caller history through payload

Suppose Node A calls Node B, and Node B calls Node C. Node C's immediate caller is Node B. If Node C also needs to know that Node A was one step earlier, Node B can pass Node A in the payload. This pattern can extend as far as the Node authors want. The GNC does not need to store the whole chain for everyone.

18. External Calls and External Contracts

Governance does not live only inside the GNC. A Node may need to upgrade a proxy, move treasury funds, call a token contract, read an oracle, update a registry, or send a cross-chain message. The GNC therefore supports external reads through Conditions and external calls through Actions.

18.1 Read-only external calls in Conditions

A Condition may read values from another smart contract by making a read-only call. The target contract must expose the needed value through a callable function. If it does not, an adapter contract can expose it. This is how a Node can check a token balance, oracle price, vote tally, delegation registry, or any other on-chain fact.

18.2 State-changing external calls in Actions

A Call Action may call any external contract with arbitrary calldata and value, subject only to EVM rules and available gas. This is intentionally powerful. It lets governance do real things. It also means a badly designed Node can do dangerous things.

The GNC should not globally restrict this. A safer Node template can restrict targets by checking target addresses in Conditions before making calls. A project's own Governor Node can include an allowlist in Data. But the base GNC should not impose one universal policy.

18.3 How external contracts should trust Nodes

An external contract that wants to recognize a Node as an authority should check both the GNC address and the exact NodeRef. For example, a proxy contract may allow upgrades only when called by the GNC and only when the calldata includes the authorized Governor NodeID.VersionPath.

```
External proxy rule, conceptually:
require(msg.sender == GNC_ADDRESS)
require(nodeRef == AuthorizedGovernorNodeID.VersionPath)
perform upgrade
```

This is more precise than trusting a wallet or trusting only a NodeID. If the Governor Node forks, the proxy can decide whether to trust the new VersionPath. Nothing is automatic unless the proxy's own logic makes it automatic.

18.4 Formal verification boundary for calls

The GNC can make the authorization logic visible and tractable. It can show that a Call Action was executed only because a Clause's Condition was true. It cannot prove what an arbitrary external contract will do once called. External targets remain their own verification problem.

19. Forking, Reuse, and Credit

Forking is not an afterthought. It is one of the main reasons Governance Nodes should exist. Good governance rules should become public goods. When a good Node exists, other projects should be able to copy it, adapt it, and retain visible lineage.

19.1 Forking behavior

When createNode is called in FORK mode, the GNC copies the source Node under the same NodeID and assigns a new child VersionPath. The new Node is independent. It has its own Data and Clauses. It records parent lineage in Identity.

```
Source: Node 100.4
Existing children: 100.4.1, 100.4.2
Next fork: 100.4.3
```

19.2 Recreating versus forking

A user can always create a new Node from scratch that happens to mirror an existing Node. The GNC cannot stop that. But the culturally correct way to build on an existing Node is to fork it. Forking preserves credit and lets tools show lineage. Recreating without lineage is the governance equivalent of copying an academic idea without citation.

19.3 Templates

A template is not a special on-chain type. It is simply a Node that people commonly fork or reference. A quorum template, veto template, token-vote template, or timelock template can all be ordinary Nodes. The GNC does not need a template registry. Communities, interfaces, and indexers can surface useful template Nodes socially.

20. The View Function

Rather than exposing many specialized getters, the GNC should expose one generalized view function. The query tells the GNC what the caller wants.

```
viewNode(nodeRef, query) -> bytes
```

Examples of query targets include:

- Identity full record;
- Data full record;
- Data at a specific path;
- Clauses full list;
- one Clause by clauseId;
- Actions inside a Clause;
- lineage metadata;
- human-readable name and description;
- full Node export.

The exact query encoding is an implementation detail, but it should be documented and stable. UIs and indexers will rely heavily on viewNode.

21. Events and Logs

Events make the GNC observable. A public governance substrate needs strong event hygiene because most users will not read raw storage. Indexers, explorers, dashboards, Discord bots, governance frontends, and audit tools will use events to understand what happened.

Event	When emitted
NodeCreated	A new NodeID.VersionPath is created.
NodeForked	A Node is copied into a new VersionPath.
NodeEvaluated	Optional event for non-view evaluation contexts, if supported.
NodeEnacted	enactNode completes successfully.
ClauseEvaluated	A Clause is evaluated, optionally only in verbose mode.

ActionExecuted	An Action succeeds or fails with SKIP.
NodeEdited	An Edit Action changes the current Node.
ExternalCallExecuted	A Call Action calls an external contract.
ReturnValue	A Return Action appends a return record.
NodeLog	An Emit Action emits Node-defined log data.

Implementations do not need to emit every possible diagnostic event by default because events cost gas. A practical implementation can provide concise default events and allow Nodes to emit their own detailed logs through Emit Actions.

22. Running Example: Treasury Grants and Protocol Upgrade Governance

This example is richer than a conference-stage proxy-upgrade example, but still small enough to follow. Imagine a protocol called Atlas DAO. Atlas wants one governance system that can approve grants from its treasury and upgrade its core protocol proxy. It wants token voting, delegate support, a veto window, a timelock, and reusable rules that other projects can fork.

22.1 The cast of Nodes

Informal name	What the Node stores / does
Atlas Governor	Top-level Node that decides whether a proposal may execute.
Proposal Node	Stores proposal data: target, calldata, grant amount, vote results, status.
Voter Eligibility Node	Defines who counts as an eligible voter or delegate.
Vote Result Node	Reads vote tallies and returns whether threshold is met.
Veto Node	Stores whether a veto was submitted during the veto window.
Timelock Node	Returns true only after the required delay has passed.
Grant Action Node	Encodes the treasury transfer call.
Upgrade Action Node	Encodes the proxy upgrade call.

None of these are enforced types. They are ordinary Nodes used in different roles.

22.2 Proposal Node Data

```
ProposalNode.Data = {
  proposalKind: "GRANT" or "UPGRADE",
  target: address,
  calldata: bytes,
  amount: uint256,
  voteStart: timestamp,
  voteEnd: timestamp,
  yesVotes: uint256,
  noVotes: uint256,
```

```

    vetoStart: timestamp,
    vetoEnd: timestamp,
    passed: bool
}

```

The exact encoding is implementation-defined. The important part is that Conditions can reference these fields and Actions can edit them.

22.3 Vote Result Node

The Vote Result Node can be a reusable rule. It may say: IF yesVotes are greater than noVotes and yesVotes meet quorum, THEN return TRUE.

```

IF (
  AND (
    GT (PAYLOAD.yesVotes, PAYLOAD.noVotes),
    GTE (PAYLOAD.yesVotes, Self.Data.quorumRequired)
  ),
  THEN (RETURN (TRUE))
)

```

Atlas can fork this Node if it wants to adjust quorum. Another DAO can fork it too, keeping lineage visible.

22.4 Veto Node

The Veto Node can store whether a valid veto has been submitted. It might allow a council address or another governance Node to set vetoed = TRUE during a window. The Governor Node does not directly edit the Veto Node. It reads it.

```

Governor condition includes:
EQ (Node[VetoNode.1].Data.vetoed, FALSE)

```

22.5 Timelock Node

The Timelock Node can return TRUE only after a proposal-specific timestamp. It can be reusable because many governance systems need the same delay pattern.

```

IF (
  GT (BLOCK_TIMESTAMP, PAYLOAD.unlockTime),
  THEN (RETURN (TRUE))
)

```

22.6 Atlas Governor Node

The Atlas Governor Node composes the others. It might have a Clause like this:

```

IF (
  AND (
    EQ (Node[VoteResult.1] (payload), TRUE),
    EQ (Node[VetoNode.1].Data.vetoed, FALSE),
    EQ (Node[Timelock.1] (payload), TRUE),
    EQ (Node[ProposalNode.3].Data.passed, TRUE)
  ),
  THEN (
    CALL (PAYLOAD.target, PAYLOAD.calldata),

```

))

The syntax above is illustrative. The underlying implementation may represent Node evaluation and payload passing differently, but the governance idea is clear: the Governor does not contain every rule inline. It composes rules by referencing other Nodes.

22.7 External proxy authorization

If the Governor is allowed to upgrade a proxy, the proxy should require that the call come from the GNC and include the exact authorized Governor NodeRef. For example, AtlasProxy may accept upgrades only if:

```
msg.sender == GNC_ADDRESS
nodeRef == AtlasGovernor.2.1
```

If Atlas later forks the Governor to AtlasGovernor.2.2, the proxy does not automatically trust it. Atlas must decide whether and how the proxy should recognize the new version. This is a feature. Authority should be explicit.

23. Formal Verification

Governance Nodes are designed to make governance logic more formally tractable. A Node has three fields. Conditions use twelve operators. Actions use four action types. This is far easier to reason about than arbitrary governance bytecode spread across many contracts.

23.1 What can be verified

- The GNC execution engine itself can be verified against this specification.
- The self-editing rule can be verified: a Node cannot directly edit another Node.
- The operator semantics can be verified.
- The action dispatch rules can be verified.
- A specific Node can be checked for properties such as: this call can only happen if these Conditions are true.
- A specific Node can be checked for reachability or stuck states, subject to bounded assumptions about external reads and calls.

23.2 What cannot be automatically verified

- The behavior of arbitrary external contracts called by a Node.
- Off-chain facts unless they are represented on-chain through verified contracts or oracles.
- Social meaning of Data schemas unless those schemas are specified.
- Safety of a Node that intentionally creates circular logic or unbounded recursion.
- Whether a community should trust a particular Node.

This boundary should be stated honestly. The GNC makes the authorization process visible. It does not magically make every target of governance safe.

23.3 Tradeoff: arbitrary calls versus full verification

The biggest verification tradeoff is the Call Action. Full governance expressiveness requires arbitrary calls. Arbitrary calls limit what the GNC can prove about outcomes. The spec chooses expressiveness. Verification tools can still

prove a great deal about when calls are authorized, but they cannot prove arbitrary external effects without also modeling the target contracts.

24. Vyper Implementation Guidance

This section is implementation guidance, not a separate protocol. The GNC should be implemented in a way that preserves the conceptual rules above while respecting Vyper and EVM realities.

24.1 Storage identity

Never use an array index as a canonical Node identity. Node identity is `NodeID.VersionPath`. Storage may use mappings from encoded `NodeRef` to records. Clause order may use indexes, but `clauseId` should be stable and stored separately.

24.2 VersionPath encoding

A `VersionPath` is a list of integers. On-chain, the implementation may encode it as bytes, as a fixed-depth array, or as a hash plus stored path records. The encoding must preserve ordering and uniqueness. Tools must be able to display the canonical dotted path.

A practical mapping pattern is to store a version node as a path hash and separately store parent path and child counters. For example, each `NodeID + parentVersionPath` can have a `nextChild` counter. Forking increments that counter and creates the child path.

24.3 Data storage

Vyper implementations should prefer explicit mappings and typed records over nested dynamic structures that are hard to copy. A field path can be represented by a `bytes32` field key. A value can be represented by a type tag and encoded bytes. Large data can be chunked or stored through references if needed.

The conceptual spec does not impose hard size limits. A reference implementation may impose practical limits because Vyper types and gas costs require them. If limits exist, they should be documented as implementation limits, not as governance philosophy.

24.4 Clause and Action storage

Clauses and Actions may be stored in mappings keyed by `NodeRef` and `clauseId/actionId`. Order can be stored separately. This avoids the need to move large arrays when a Clause is inserted, removed, or reordered. It also helps keep `clauseId` stable.

24.5 Deep copy on fork

Forking must deep-copy mutable Node contents. The copied version may share immutable content-addressed chunks if the implementation supports them, but it must not share mutable records in a way that allows parent edits to affect the child or child edits to affect the parent.

24.6 Gas as practical constraint

The conceptual protocol does not define `MAX_CLAUSES` or `MAX_ACTIONS`. The EVM still imposes gas limits. Vyper may impose type bounds. A reference implementation can choose storage patterns that make common cases efficient, but authors are responsible for not creating Nodes that are too expensive to enact.

24.7 Suggested internal enums

Enum	Values
Operator	=, !=, <, >, <=, >=, IN, CONTAINS, IS_DEFINED, AND, OR, NOT
ActionType	RETURN, CALL, EDIT, EMIT
OnFailure	SKIP = 0, REVERT = 1
ValueType	BOOL, UINT, INT, ADDRESS, BYTES32, BYTES, STRING, NODE_REF, LIST, STRUCT or implementation-defined extensions

25. Edge Cases and Failure Modes

A good spec should not only describe the happy path. This section lists important edge cases and the expected behavior.

Edge case	Recommended behavior
NodeRef does not exist	Condition referencing it fails or reverts according to implementation; direct evaluate/enact should revert with clear error.
VersionPath malformed	Reject the call. VersionPath must be canonical.
Data field undefined	IS_DEFINED returns false. Other operators on missing data should fail the Condition or revert with diagnostic behavior.
Type mismatch	Condition fails or reverts; implementation must document the rule. Strict failure is recommended.
Multiple Return Actions	Return all records in order, each with clauseId.
Action fails with SKIP	Continue to next Action. Emit status if possible.
Action fails with REVERT	Revert entire enactNode transaction.
Clause edits a later Clause	Under live sequential semantics, later Clause is read when reached and may observe the edit.
Clause edits itself	Current execution continues from already-loaded action; future executions use edited Clause. Implementation must document exact behavior.
External call reverts	Treat as Action failure and apply onFailure.
External call reenters GNC	Allowed unless implementation imposes a guard; behavior must be explicit and tested.
Recursive Node calls run out of gas	Transaction fails due to gas. This is an authoring problem, not a protocol violation.
Fork while parent later changes	Fork remains independent. Parent changes do not affect child.
Human-readable name collision	Allowed. Names are not canonical.
Attempt to edit NodeId or VersionPath	Reject. These fields are GNC-controlled.

25.1 Stuck Nodes

A Node can become stuck. It may remove its own ability to edit. It may set Conditions that can never be true. It may depend on a deleted external contract. This is acceptable. Ethereum already allows immutable contracts that become unusable. The GNC should not add global rescue powers to prevent this. Communities should use safer templates if they want escape hatches.

25.2 Cycles

Nodes may reference one another cyclically. A cycle may be harmless if it only reads stable Data. It may be dangerous if it creates recursive enactment loops. The GNC does not need to forbid cycles. Gas is the practical constraint. Tooling should detect and warn about cycles, but the base contract should not forbid all of them because some cyclic references may be meaningful.

25.3 Same-block ordering

Transactions in the same block execute sequentially in builder-chosen order. The GNC does not need a special same-block lock unless an implementation chooses to add one. Conditions are evaluated against the state visible when that transaction executes. Authors who need stronger protection against stale assumptions can include `lastEditedBlock` or other fields in Conditions.

26. Testing Checklist

A reference implementation should include a large test suite. The following checklist is a starting point.

- Create a new Node and verify Identity, Data, and Clauses are stored correctly.
- Fork a Node and verify the VersionPath and parent lineage are correct.
- Fork a Node, edit the parent, and verify the child does not change.
- Fork a Node, edit the child, and verify the parent does not change.
- Attempt to edit NodeID and VersionPath and verify rejection.
- Evaluate simple Clauses using each of the twelve operators.
- Evaluate nested formula Conditions such as `OR(AND(A, B), C)`.
- Evaluate Conditions that read another Node by `NodeID.VersionPath`.
- Evaluate Conditions that read external contract view functions.
- Enact Clauses in order and verify all true Clauses execute.
- Verify multiple Return Actions return structured records with `clauseId`.
- Verify Action failure mode `SKIP` continues.
- Verify Action failure mode `REVERT` reverts the whole enactment.
- Verify Edit Actions can edit Data, Clauses, and editable Identity metadata of the same Node.
- Verify Edit Actions cannot edit another Node directly.
- Verify Call Actions can call another Node and that the target Node sees caller as the calling Node.
- Verify Call Actions can call arbitrary external contracts with arbitrary `calldata`.
- Test reentrancy scenarios deliberately.
- Test undefined fields and `IS_DEFINED`.
- Test malformed VersionPaths.
- Test name collisions.
- Test gas-heavy Nodes to understand practical limits.
- Fuzz Condition encodings and Action encodings.
- Fuzz fork/copy behavior to detect aliasing or shallow-copy bugs.

- Simulate recursive Node calls and verify failure is understandable.
- Test event emission and off-chain indexability.

27. Reference Implementation Milestones

A sensible build path is incremental. Do not start by trying to implement every possible Data schema and external call shape. Build the smallest correct GNC, then expand.

29. Implement Node Identity and VersionPath storage.
30. Implement createNode in NEW mode.
31. Implement viewNode for Identity, Data, and Clauses.
32. Implement simple Data values and literal Conditions.
33. Implement the twelve operators over basic types.
34. Implement evaluateNode for local Data only.
35. Implement Return and Emit Actions.
36. Implement Edit Actions for Self.Data.
37. Implement Edit Actions for Self.Clauses.
38. Implement enactNode with ordered Clause and Action execution.
39. Implement Action failure modes SKIP and REVERT.
40. Implement NodeRef reads from other Nodes.
41. Implement Call Actions to GNC evaluateNode/enactNode.
42. Implement external read Conditions.
43. Implement external Call Actions.
44. Implement createNode in FORK mode with deep copy and VersionPath child assignment.
45. Build the Atlas-style running example as an integration test.
46. Add fuzzing, invariant tests, and gas profiling.
47. Publish the reference implementation and invite audits.

28. Glossary

Term	Definition
Action	An operation executed when a Clause Condition is true. v1 has Return, Call, Edit, and Emit.
CALLER	The immediate entity calling a Node: wallet, contract, or NodeRef.
Clause	An IF/THEN rule inside a Node.
Condition	A formula-like boolean expression inside a Clause.
Data	Arbitrary structured values stored inside a Node.
Emit	An Action that emits an event/log.
Enact	To evaluate a Node and execute Actions for true Clauses.
Evaluate	To read a Node and compute Clause results without changing state.
Fork	To copy a Node into a new VersionPath under the same NodeID.
GNC	Governance Nodes Contract, the single immutable contract specified here.

Identity	Node metadata including NodeID, VersionPath, lineage, creator, and timestamps.
Node	The primitive governance object stored in the GNC.
NodeID	Numerical canonical identifier for a Node lineage.
NodeRef	A full reference to a specific Node version: NodeID.VersionPath.
Payload	Freeform bytes supplied with evaluateNode or enactNode.
Self-editing rule	A Node can edit itself through its Clauses but cannot directly edit another Node.
VersionPath	Ordered integer path showing a Node version and fork lineage.

29. Authoring Nodes Like Spreadsheets

A Governance Node should feel less like writing a new contract and more like defining a living spreadsheet rule. A spreadsheet user does not think in terms of bytecode, stack frames, or opcodes. They think: if this cell is greater than that cell, then show this value. Governance Nodes use the same human pattern, but with on-chain state and on-chain consequences.

The author-facing shape of a Clause is therefore IF(condition, THEN(actions)). The Condition can be a nested formula. The Actions can return values, make calls, edit the Node, or emit logs. Tooling should let authors write formulas in a familiar style, validate those formulas off-chain, and compile them into the canonical on-chain encoding accepted by the GNC.

This matters because the GNC is not only for protocol engineers. Protocol engineers will implement it, audit it, and build the first Nodes. But over time, governance designers, DAO operators, lawyers, treasurers, delegates, and researchers should be able to read Nodes. They may not write raw Vyper, but they can understand IF quorum is met AND veto is false AND timelock expired, THEN execute proposal. The system should invite that level of literacy.

The canonical contract should not parse human text directly. On-chain text parsing is expensive and fragile. Instead, the ecosystem should develop authoring tools that present Nodes in an Excel-like formula format and compile those formulas into deterministic structured data. The formula view is for people. The encoded tree is for the GNC.

A good authoring interface should show at least three views of the same Clause: a friendly formula view, a structured tree view, and a raw encoded view. The friendly view helps humans understand the rule. The tree view helps auditors reason about precedence and nesting. The raw view helps implementers verify that the bytes stored on-chain match the intended rule.

The GNC should not require authors to use one official editor. It should only define the canonical encoding and behavior. Competing interfaces can then emerge. Some may look like spreadsheets. Some may look like node graphs. Some may look like low-code workflow builders. Some may be command-line tools for developers. The contract should be neutral among them.

The key authoring rule is that everything visible in the friendly interface must compile to the same Node fields: Identity, Data, and Clauses. If an interface invents new concepts, those concepts are interface-level conveniences. They do not become new top-level fields in the GNC. This keeps the base contract stable while letting tooling evolve quickly.

30. Detailed Clause Authoring Patterns

This section gives several common patterns in plain English. These are not special cases in the contract. They are examples of how ordinary Clauses can express familiar governance behavior.

A simple approval flag is the smallest useful pattern. The Node starts with `Data.approved = FALSE`. It has a Clause saying `IF CALLER = approvedSigner, THEN EDIT Self.Data.approved = TRUE`. A second Clause may say `IF Self.Data.approved = TRUE, THEN RETURN TRUE`. This Node can now be referenced by other Nodes as an approval condition.

A one-way latch is a Node that can turn on but not turn off. It can be built without adding a special latch field to the GNC. The Node has a Clause saying `IF some condition is true, THEN EDIT Self.Data.state = TRUE` and `EDIT Self.Clauses` to remove or disable the Clause that would change it again. Once enacted, the Node has changed itself into a new shape that no longer contains the path back. This is powerful and dangerous, exactly like immutable smart contract design generally.

A timelock is a Node that stores an unlock timestamp in `Data`. Its Clause says `IF BLOCK_TIMESTAMP > Self.Data.unlockTime, THEN RETURN TRUE`. A more elaborate timelock can include a start action, a cancel path, or multiple phases. Those are not new primitives. They are just more Clauses.

A veto is a Node that stores `vetoed = FALSE`. During a veto window, an authorized caller can enact a Clause that sets `vetoed = TRUE`. Other Nodes can reference the veto Node by checking `EQ(Node[VetoNode].Data.vetoed, FALSE)`. The veto Node owns its own veto logic. The Governor Node merely reads it.

A proposal status Node may store `yesVotes`, `noVotes`, `quorum`, `deadline`, `passed`, and `executed`. Its Clauses can update `passed` when the vote threshold is met, update `executed` after a successful call, and emit events for off-chain tools. Another Node can read only the final `passed` field, or it can inspect the vote fields directly.

A reusable condition can be created by placing a formula in a Node whose main purpose is to return `TRUE` or `FALSE`. Other Nodes can call `evaluateNode` on it or read its `Data` if the condition writes its output there. This is how a community can build a library of useful governance rules without giving those rules a special type at the contract level.

A reusable action can be created by placing `calldata` and target information in `Data` and exposing a Clause that calls it when authorized. Other Nodes can call this action Node. The action Node still governs itself; it decides whether the caller and payload satisfy its own Clauses before it acts.

A social-process Node can store human-readable commitments, off-chain references, or deliberation state in `Data`. The GNC does not care whether the `Data` is a strict machine schema or a text blob. If another Node wants to reason about it, the relevant parts need to be machine-readable. If it is only for humans, it can remain human-readable.

An emergency shutdown Node can be built as a normal Node. It might allow a small council to set `Data.shutdown = TRUE`, and other Nodes can include `NOT(Node[Shutdown].Data.shutdown)` in their Conditions. There is no global pause switch in the GNC. If a system wants a pause switch, it composes one from Nodes.

31. Stale State, Ordering, and Ethereum Reality

Smart contracts do not run in a magical governance vacuum. They run in the Ethereum execution model. Transactions are ordered inside blocks. State changes from earlier transactions in a block are visible to later transactions in that same block. Mempool timing, block builder ordering, reverts, and gas limits all matter. The GNC should not pretend otherwise.

The GNC uses `NodeID.VersionPath` for identity and lineage, not as a mutation counter. A Node can evolve without receiving a new `VersionPath`. Therefore, if a caller wants to ensure that a Node has not changed since the caller last

inspected it, the Node or payload should include an explicit Condition that checks the relevant lastEdited metadata or Data field. The base GNC does not force every enactment to include an expected revision number.

This is a deliberate simplification. Some systems prefer optimistic concurrency controls. Governance Nodes can express those controls as Clauses. For example, a Node can require `EQ(Self.Identity.lastEditedBlock, PAYLOAD.expectedLastEditedBlock)` before accepting a sensitive edit. Another Node may not care. The base GNC should not burden the simple case with a rule only some systems need.

The same principle applies to one-edit-per-block behavior. Some systems may want it. Some may not. A Node can include a Condition that checks lastEditedBlock if it wants to reject multiple edits in the same block. Another Node may intentionally allow multiple changes in the same block. The GNC should give Nodes access to the facts and let their Clauses decide.

Ordering inside enactNode is deterministic: Clauses are evaluated in order, and Actions inside a true Clause execute in order. Ordering across transactions is determined by Ethereum. If two users try to enact the same Node at the same time, whichever transaction executes first sees the old state, and the later transaction sees the new state. That is not a race condition in the contract. That is how Ethereum works.

Authors should think carefully about Actions that call external contracts before editing internal state. If the external contract calls back into the GNC, it may observe the Node before the intended edit. Conversely, if the Node edits itself first and then calls externally, a later failure may revert everything if `onFailure = REVERT`, or may leave the edit in place if `onFailure = SKIP` and the failure is handled. These are not bugs if they are specified and tested. They are governance design choices.

A useful authoring habit is to make important Clauses explicit about the state they expect. If a proposal should only execute once, include a Condition requiring `executed = FALSE` and an Action that sets `executed = TRUE`. If a vote tally should only be accepted before a deadline, include the deadline in the Condition. If an external contract should only be called by one specific Node version, make that external contract verify the exact NodeRef.

The GNC should expose enough metadata for Nodes to write these rules: lastEditedBlock, lastEditedTime if available, caller, payload, block number, block timestamp, and exact NodeRef. It should not decide every policy globally.

32. Common Builder Questions

Why not just use existing DAO frameworks?

Existing frameworks are useful, but they usually package governance as contracts, plugins, or predefined flows. Governance Nodes tries to make governance logic itself into reusable on-chain state. The point is not to replace every tool on day one. The point is to create a lower-level substrate that many tools can build on.

Is the GNC a programming language?

Not in the broad sense. It is a finite governance instruction system. It has a small set of operators and action types. Human-facing tools may make it feel like a formula language, but the contract is not trying to be Solidity inside Solidity.

Can a Node store text?

Yes. Data can store human-readable text if the implementation supports strings or bytes. The GNC does not care whether a Data field is for machines, humans, or both. Conditions can only reason about data that is encoded in a way the GNC understands.

Can a Node call any contract?

Yes. Call Actions are intentionally unrestricted at the base layer. A Node that wants restrictions can encode them in its own Clauses.

Can a Node edit another Node?

No, not directly. A Node can call another Node, and the target Node can then edit itself if its own Clauses allow it.

Can a Node edit its own Clauses?

Yes. That is how governance over governance is represented. The Node changes itself through its own rules.

Can a Node break itself forever?

Yes. This is part of immutability and maximum expressiveness. Safer templates can include recovery paths, but the base GNC should not impose one.

What is the difference between a fork and an edit?

An edit changes the same NodeID.VersionPath over time. A fork creates a new VersionPath under the same NodeID, preserving lineage and creating an independent copy.

Why preserve NodeID when forking?

Because forking is a citation and lineage mechanism. It shows that the new Node descends from the old one. This matters for credit, research, reuse, and public-good culture.

Can someone copy a Node into a fresh NodeID anyway?

Yes. The protocol cannot prevent someone from recreating logic manually. It can only make proper forking easy and visible.

Why are Node names not unique?

Human-readable names are for people. Canonical references are numerical NodeID plus VersionPath. Two Nodes can have the same name without ambiguity because their NodeRefs differ.

Why exactly twelve operators?

The twelve operators cover the core boolean and comparison logic needed for governance: equality, inequality, ordering, membership, existence, and boolean composition. More can be added only by deploying a different GNC or by encoding richer behavior through calls and data.

Why exactly four Action types?

Return, Call, Edit, and Emit are enough to express a large space of governance behavior. Return communicates, Call interacts, Edit changes the Node, and Emit records. Complex workflows are compositions of these four.

What does evaluateNode do with Call Actions?

evaluateNode does not perform state-changing Actions. It can report that a Call Action would be reached if the Clause is true, and it can return Return values, but it must remain read-only.

What happens if multiple Clauses return values?

All return records are included in order. Each record includes the clauseId that produced it.

Why not include a global pause button?

A global pause button requires a global authority or global rule. The GNC is neutral. If a governance system wants a pause mechanism, it can build one as a Node and have other Nodes reference it.

Why not include a global allowlist of safe external calls?

Same reason. Safety policies should be local to Nodes and communities, not hardcoded into the base contract.

Can the GNC be formally verified?

The GNC execution engine and operator/action semantics can be formally verified. Individual Nodes can be analyzed. Arbitrary external contracts called by Nodes must be verified separately.

Why Vyper instead of Solidity?

The reference implementation should minimize accidental aliasing, shallow-copy mistakes, and pointer-like confusion when Nodes are copied, edited, or cleared. Vyper pushes the design toward explicit, auditable storage behavior.

Does gas limit the complexity of Nodes?

Yes. The conceptual spec does not define universal hard caps, but the EVM gas limit and implementation choices impose practical constraints.

Should tooling warn users about dangerous Nodes?

Absolutely. The base contract should be neutral, but frontends, linters, auditors, and verification tools should warn loudly about circular calls, impossible Conditions, dangerous external calls, and self-locking edits.

Can Nodes exist on multiple chains?

The spec is EVM-oriented. A GNC can be deployed on any EVM-compatible chain. Cross-chain references are not part of v1, but Nodes can call bridge contracts or store cross-chain metadata in Data.

Who owns a Node?

There is no universal owner field in the base schema. Authority is expressed by Clauses. A Node may store creator or maintainer metadata, but control comes from IF/THEN rules, not from a built-in owner.

Is Data required to be machine-readable?

No. But if Conditions or Actions need to use it, those parts must be encoded in a machine-readable way. Purely descriptive Data can remain freeform.

What should the first implementation prioritize?

Correctness over cleverness. Implement the minimal schema, create/evaluate/enact/view, basic Data values, the twelve operators, the four Actions, forking with deep copy, and strong tests before adding fancy authoring features.

Closing Note

The GNC is small in shape but large in implication. It asks a simple question: what if governance logic did not need to live in bespoke bytecode? What if governance rules could be public, composable, forkable state? What if the useful parts of one governance system could become building blocks for the next?

This specification is the starting point. The first implementation will discover details the spec cannot predict. That is expected. The important thing is to preserve the core: one immutable GNC, Nodes with Identity/Data/Clauses, IF/THEN Clauses, twelve operators, four Actions, versioned lineage, self-editing sovereignty, and enough freedom for people to build governance systems nobody has imagined yet.

For it is not death we truly fear, but something much more aghast

To be forever relegated as but a footnote to Plato

So, what say we, to the Philosopher Kings?

Not today